



Backing Up Apache Iceberg Tables Across Environments

Project Nessie as the control plane, Apache Beam as the engine

Vineel Arekapaudi

ICEBERG + NESSIE

CONTROL-PLANE MIRRORING

POWERED BY APACHE BEAM

The Why



Fun challenge (or not!) Try taking a backup and restore a SQL Server instance using Delphix

→ 68 QA and Dev database environments spanning multiple instances across Dev, QA, and QA, and Pre-Prod.

→ Developers, QA engineers, and Market Risk officers need reliable, consistent data snapshots data snapshots for simulations, audit, and compliance purposes.

→ This is the production workflow we use to solve that problem with Apache Iceberg and Iceberg and Project Nessie.

Iceberg Is Not Just Files

An Iceberg table = object storage + catalog state.

What you copied

- Parquet files
- Manifest lists
- Metadata JSON

What you missed

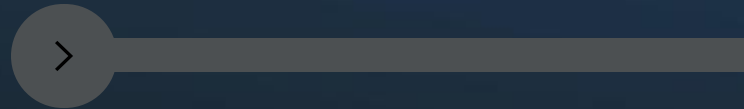
- Branch heads
- Commit hashes
- Snapshot lineage



Copying files $\neq$ mirroring an Iceberg table.

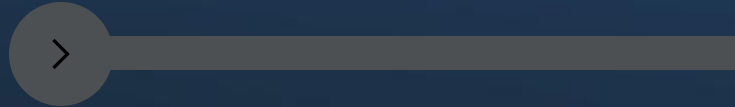
What Makes Iceberg Different

Iceberg tracks history, not just data.



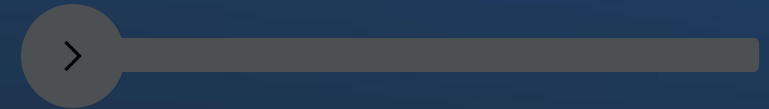
Snapshot Model

Every write creates a new snapshot. Old snapshots remain until explicitly expired.



Manifest Tree

Snapshots point to manifest lists → manifests → data files. The tree is the table.



Catalog = The Root

The catalog holds the pointer to the current snapshot. Without it, the tree has no root.

Restore without the catalog pointer = orphaned files.

The What

Enter Project Nessie

Nessie is a Git-like catalog for Iceberg with branches, tags, and commits for your data.



Branches

Isolate environments. Prod, pre-prod, and non-prod each get their own branch head.



Commits

Every table change is a commit hash. Point-in-time restore means pointing to a hash.



Tags

Mark known-good states. Tag before a migration, restore by name.

Nessie makes the catalog a first-class replication target.

The What

Storage Backend Interface:

IN_MEMORY: Ephemeral, non-persistent

ROCKSDB: Embedded, Single Node

JDBC2: Relational Databases

MONGODB2: Document Database

JDBC-based storage (PostgreSQL, MariaDB, etc.)

CASSANDRA2: Distributed NoSQL

The What

Version Control Core

- Reference - A pointer to a specific commit (either a branch or tag)
- Commit - A consistent snapshot of the repository at a point in time
- Content - The actual data objects being versioned (tables, views, etc.)
- Content Key - Unique identifier for a content object
- Operations - Actions performed on the repository (put, delete, merge, transplant)

The Two-Layer Backup Model

You need to replicate two things, and they must stay in sync.

Data Plane - Object Storage

Parquet files, manifests, metadata JSON. Replicated with rclone, distcp, or native S3 replication. **Paths preserved.**



Control Plane - Nessie Catalog

Branch heads, commit hashes, tags. The Git-like metadata that gives the files meaning.

Miss either layer and your restore story has a gap.

The 3-Environment Target State

Prod → Pre-Prod → Non-Prod. Each environment is a faithful, independently resolvable mirror.



Prod

Authoritative source. All writes happen here. here. Nessie holds the canonical branch heads. heads.



Pre-Prod

Staging mirror. Receives replicated data and and catalog state. Used for release validation and validation and DR rehearsal.



Non-Prod

Dev/test mirror. Receives the same replication. Read-only during validation windows.

It's the same data with commit lineage across three environments.

What Each Environment Gets

Each layer has a distinct role, and each one follows a distinct replication cadence.

Prod

Source of truth. No inbound replication, data is populated through daily batch/ad-hoc batch/ad-hoc changes. Nessie writes are authoritative and its GC policies set here.

Pre-Prod

Full mirror of prod data + catalog. Synced on a scheduled cadence (e.g., nightly or nightly or per-release). Used for: release testing, DR drills, audit validation.

Non-Prod

Subset or full mirror, synced less frequently. Used for: dev testing, schema experiments, query tuning.

What This Workflow Sets Out to Prove

Three behaviors, three real-world scenarios, in our production-validated workflow.

Blank-Slate Fidelity

We copy the entire catalog to a lower environment and it resolves correctly from scratch.

Incremental Propagation

We sync only changed metadata documents after the initial copy, without a full re-import.

Branch-Level Isolation

We create new branches in lower environments without disturbing the existing mirrored lineage.

Branch Isolation Across 3 Environments

Each environment has its own branch head, so they can diverge safely.

1

Prod: main

Points to the latest production commit. Never
Never written to from lower environments.
environments.

2

Pre-Prod: main (mirror)

Points to the last synced prod commit. May lag
by one sync cycle. Used for release gate
checks.

3

Non-Prod: main (mirror) + feature
branches

Points to a synced commit. Dev teams can
can create feature branches off the mirror
mirror without affecting prod lineage.

Branch heads are the levers. Move them deliberately.

Attempt #1



The intuitive first approach: if Nessie keeps its state in MongoDB, why not just copy the database?

- Replicate the `objs2` and `refs2` collections from production Mongo into the non-prod Mongo. prod Mongo.
- Pair that with `rdone` or `distcp` for the Parquet, manifest, and metadata files in S3.
- It works in a demo. Then it meets scale, version upgrades, and audit requirements.

Replication Flow: Prod → Pre-Prod → Non-Prod

Two hops. Two sync jobs. One consistent lineage across all three environments.

Prod writes

Iceberg commits land in prod S3 + prod Nessie (MongoDB refs2/objs2)

Prod → Pre-Prod sync

Object replication (S3) + MongoDB collection copy. Pre-prod Nessie branch heads updated via API.

Pre-Prod → Non-Prod sync

Same pattern. Non-prod receives from pre-prod, not directly from prod. Keeps prod. Keeps blast radius contained.

Validate

Query each environment. Branch heads should resolve to the same commit hash.



Non-prod never touches prod directly.

Workflow Setup: What Was Assumed

Storage replication was already in place through rclone. This workflow focuses on catalog fidelity.



Environments

Prod was the authoritative source. Non-prod prod was the mirror target. Same object paths in both buckets, so no metadata rewriting was needed.



What Was Copied

- **Object layer:** Parquet files, manifests, metadata JSON
- **Catalog layer:** MongoDB collections `objs2` and `refs2`
- **Runtime:** Container config and pod restart



Three Strategies Executed

- **Full copy:** Blank slate
- **Incremental copy:** By z-value and timestamp
- **New branch creation:** Isolation check

Step 1: Blank-Slate Copy

Export the entire `objs2` collection into the target et. `main` resolves correctly.

Export

Dump the full `objs2` collection from prod MongoDB.

Import

Load into non-prod MongoDB. Point the non-prod Nessie instance at the new collections.

Validate

Query the non-prod Nessie API. Confirm all namespaces and tables on `main` are present and resolvable.

Outcome: Non-prod resolves the same table state as prod. ✓

Step 2: Incremental Sync

Don't re-import everything. Copy only what changed since the last sync.

01

Find the baseline

Identify the latest commit in the target by z-value or timestamp.

02

Export the delta

From prod MongoDB, export only objs2 documents created or updated after that marker.

03

Import the delta

Load the changed documents into the target objs2 objs2 collection.

04

Update refs2

Sync the branch head in refs2 to point to the new latest commit hash.

Outcome: Non-prod advances to the newer table state without a full re-import. ✓

Incremental Sync: The Operational Playbook

Keep non-prod in sync without a full re-import on every cycle.

1 Redirect the Nessie endpoint

Update the container config so the non-non-prod Nessie pod targets the mirrored mirrored MongoDB database. Restart the the pod to activate.

2 Establish the baseline

On first run: full export/import of `obj s2`.
Confirm `main` has the expected namespaces and tables.

3 Run incremental replays

On subsequent runs: find the latest target commit by z-value or timestamp. Export only newer source documents. Import into target `obj s2`.

Verification = query the mirrored catalog endpoint after every sync.

Where Direct Database Sync Breaks Down

Files present, catalog stale

Object replication ran but MongoDB sync didn't. Non-prod resolves an old snapshot, which looks fine until you query.

GC mismatch

Prod expired a commit but Pre-prod still points to it, will have to wait until wait until the next sync.

Non-prod wrote back

A dev job wrote to the mirrored catalog. The mirror is now contaminated, so re-import is required.

refs2 synced, objs2 not

Branch heads updated but commit objects are missing, resulting in a Nessie 404 on table resolution.

Every failure mode has the same root cause: the two layers drifted.

The Evolution



One unified Apache Beam pipeline replaces ad-hoc scripts and raw database syncs.

→ It operates at the Nessie REST API level, never on MongoDB internals.

→ The same pipeline runs in both batch and streaming modes.

→ Portable across runners: built on the Direct Runner, deployed on Spark.

Why Apache Beam?

Replacing ad-hoc scripts and fragile database syncs with one engine we can test, port, and observe.

Testable

Pipelines run locally on the Direct Runner. We validate replication logic against sample catalogs before touching production data.

Portable

The same code runs on the Spark Runner in our governed cluster. No rewrite between laptop and production.

Observable

Built-in metrics, counters, and logging replace opaque shell scripts. Every record replicated is countable and traceable.

The Unified Architecture: Two Layers, One Pipeline

A single Apache Beam pipeline coordinates both replication layers.

Layer 1 - Storage Replication

Reads Parquet, manifests, and metadata JSON from prod S3, writing to non-prod S3 with parallel fan-out.



Layer 2 - Catalog Replication

Polls the production Nessie REST API for new commits and applies branch, tag, and table changes to non-prod Nessie.

Two layers, one codebase. One pipeline keeps them in sync.

Layer 1: Storage Replication with Beam

A Beam pipeline mirrors every Iceberg file from production S3 to non-prod S3.

Read

List every Iceberg object in prod S3: Parquet data files, manifest lists, manifests, and metadata metadata JSON.

Fan Out

Beam parallelizes the copy across thousands of of tables and files, with retries and backpressure backpressure handled by the runner.

Write

Objects land in non-prod S3 at identical paths, so the catalog needs no path rewriting.

Outcome: Every file an Iceberg table points to is present in non-prod. ✓

Layer 2: Catalog Replication via the Nessie REST API

Replication driven by Nessie versioning semantics, not MongoDB internals.

01

Poll

Query the production Nessie REST API for new commits and snapshot changes since the last replicated commit.

02

Extract

Pull the relevant branch, tag, and table metadata from each new commit in order.

03

Apply

Replay those changes onto the non-prod Nessie in a Nessie in a controlled, auditable sequence.

04

Verify

Confirm each branch head resolves to the expected commit hash via the Branch Content API.

Outcome: Non-prod Nessie becomes a true, API-level mirror of production. ✓

The Nessie API: Your Control Plane Interface

Five API calls turn branch-head operations into auditable, repeatable steps.

01

Authenticate

Obtain a bearer token for the target Nessie endpoint.

02

Get Reference

Read the current branch head before any change.
Establish the pre-state baseline.

03

Branch Content

Verify what a branch resolves to after sync. The primary correctness check.

04

Diff

Compare source ref against target hash. Expose drift between environments.
environments.

05

Update Reference

The key write: retarget a branch head to a mirrored commit hash.

The API makes every branch-head move auditable, not a hidden database edit.

One Pipeline, Two Modes: Batch and Streaming

The same Beam pipeline serves two distinct operational needs.

Streaming Mode

Watches for new Nessie commits and triggers incremental replication, keeping non-prod near-current.



Batch Mode

Bootstraps a full environment or recovers to a specific point in time by time by replaying to a chosen Nessie snapshot.

Same code and same guarantees, whether keeping current or rebuilding from scratch.

Runner Portability: From Laptop to Cluster

The Beam runner abstraction was essential in our regulated environment.

Direct Runner

Develop and validate the pipeline locally against sample catalogs and buckets. Fast iteration, no cluster access required.

Spark Runner

Deploy the exact same pipeline to our governed Spark cluster for production-scale replication. No rewrite.

Why It Matters

Logic is proven before it touches regulated data, and promotion to production is a config change, not a port.

The Key Insight: Recovery Is a Metadata Operation

Once the files are there, the branch head is the only thing that needs to move.

The Flow

No object re-copy required when the target already holds the right Iceberg files.

The Key Insight

No object re-copy required - When the target already holds the right Iceberg files, only the ref pointer needs to move.

Recovery is fast and precise - A metadata operation, not a full storage restore.

Branch heads are explicit levers - For rewind, failover rehearsal, and point-in-time restore tests.

Lessons Learned in Production

Nessie API pagination

List and diff endpoints page heavily. Follow continuation tokens fully, or fully, or you silently miss data.

Failures mid-replication

A run can die between files and commits. Steps are idempotent; bad bad records route to a dead-letter queue.

Non-atomic sync

Storage and catalog never commit together. Replicate files first, then advance the branch head.

Resilient to upgrades

Working at the REST API level, not MongoDB collections, kept replication stable across Nessie upgrades.

Consistency is a discipline: files first, catalog second, verify always.

Applying This to 3 Environments: The Sync Matrix

Each environment pair has its own sync job, cadence, and validation gate.

Environment Pair	What Gets Synced	Cadence	Validation Gate
Prod → Pre-Prod	S3 + Nessie commits (API)	Nightly / per-release	Branch content API + Spark query
Pre-Prod → Non-Prod	S3 + Nessie commits (API)	On-demand / weekly	Branch content API + query engine
Non-Prod (internal)	Feature branches only	Dev-triggered	Branch diff API

Stagger the cadences. Pre-prod is your DR rehearsal. Non-prod is your sandbox.

Key Takeaways

What we would tell another regulated team starting this journey.

Mirror the catalog, not the database

Replicate Nessie through its REST API. You inherit its versioning semantics and stay resilient to upgrades.

One pipeline, two layers

Let a single Beam job own both storage and catalog so the two never drift, in batch or streaming.

Recovery is a pointer move

With files already in place, restore is a branch-head update: fast, precise, and auditable.

Thank you

Backing up Apache Iceberg across environments with Project Nessie and Apache Beam

